DBA Agent: Architecture & System Report

Engineering a State-Driven LLM Assistant for Automated Relational Database Design

COURSE	INSTRUCTOR	GRADER	IMPLEMENTATION
Database Laboratory	Dr. Farnush Manavi	Ali Bagherieh	C# (.NET) / Spectre / Scriban

1. Executive Summary & Core Philosophy

The **DBA Agent** is an autonomous, agentic system engineered to assist software teams in translating unstructured domain descriptions into fully realized, production-grade relational database architectures. Rather than delegating database creation to unconstrained end-to-end prompt generation—a practice notoriously prone to hallucinatory data types, broken foreign key ordering, and syntax anomalies—the system is designed around a *deterministic finite state machine (FSM)* that governs targeted, contextual Large Language Model (LLM) calls.

The architecture enforces strict separation between workflow orchestration, domain knowledge synthesis, and physical database validation. Built on .NET using C#, the application pairs externalized Scriban templating with an automated PostgreSQL Docker execution runner. This delivers a resilient pipeline capable of iterative requirements gathering, formal normalization, cross-artifact synchronization, and self-healing SQL execution.

2. System Architecture & Pipeline Stages

The workflow is divided into six sequential, tightly bounded phases followed by an automated live validation engine. Every stage accepts explicit state artifacts, renders isolated prompt templates, validates structured outputs, and writes canonical deliverables to disk before transitioning.

Stage	Input Artifacts	Core Responsibility	Output Deliverable
1. Intake	Raw User Prompt	Extracts key actors, primary entities, and core business processes.	`state.json` (Intake Summary)
2. Interview	Intake Model & Turn History	Generates domain queries across 7 technical categories with multi-round stopping logic.	`requirements.md`
3. DBMS Selection	Requirements Specification	Evaluates PostgreSQL, MySQL, SQLite, and SQL Server using an architectural decision matrix.	`dbms-choice.md`
4. Design Docs	Requirements & DBMS Choice	Generates conceptual, logical, normalization, and assumption models in Markdown.	`conceptual-model.md`, `logical-schema.md`, etc.
5. Revision Loop	Natural Language Change	Performs impact analysis and simultaneously updates all affected deliverables.	`changelog.md` + updated docs

Stage	Input Artifacts	Core Responsibility	Output Deliverable
6. SQL Generation	Approved Design Documents	Produces topological DDL scripts and realistic, non-placeholder test data.	`schema.sql`, `seed.sql`
Bonus (+15%)	`schema.sql` & `seed.sql`	Executes DDL/DML on a live PostgreSQL container; triggers LLM repair loop on error.	Verified Live Database + logs

Stage 2: Adaptive Requirements Interview Engine

Stage 2 implements an interactive dialogue engine covering entities, cardinalities, data volumes, read/write ratios, ACID transactions, security roles, and business invariants. To satisfy project specifications without unbounded execution costs, the engine utilizes a dual-prompt strategy: one prompt generates targeted technical inquiries, while an independent evaluation prompt parses the conversation history into a structured JSON decision payload (`{"status": "CONTINUE"|"COMPLETE", "reasoning": "..."}`). The evaluator enforces a hard minimum threshold of two rounds while checking for requirement saturation before synthesizing `requirements.md`.

Stage 5: Atomic Multi-File Revision Propagation

A common failure in multi-stage LLM pipelines is localized drift: when a user modifies a requirement (e.g., adding courier tipping), naive agents update only the logical schema while leaving the conceptual model and normalization proofs inconsistent. Stage 5 mitigates this through a two-step impact analysis:

- **Dependency Impact Analysis:** An LLM analyzer inspects the change request and returns an exact JSON array of impacted files (e.g., `["conceptual-model.md", "logical-schema.md", "normalization.md"]`).
- **Synchronous Re-Generation:** Each identified file is sequentially re-prompted with the change summary, ensuring cross-document integrity. Every edit is stamped into `changelog.md` with timestamps, user prompts, and rationale.

3. Key Design Decisions & Engineering Patterns

3.1 Externalized Scriban Prompt Templating

In accordance with Section 4 technical requirements, zero prompt text is hard-coded within compiled C# binaries. All system and user prompts reside in the `prompts/` directory as standalone `.scriban` templates. The `PromptService` employs a recursive directory resolver that inspects execution binaries, current working directories, and parent solution structures. This enables runtime prompt refinement and variable binding without recompilation.

3.2 State Persistence & Session Hydration

Session durability is managed via `StateManager`, which serializes the unified `SessionState` entity to `output/state.json` using `System.Text.Json`. The state tracks current workflow stage, completed interview turns, selected DBMS engine, approval flags, and full document contents. On startup, the CLI automatically detects prior sessions, allowing users to safely terminate the process and resume without data loss.

3.3 Provider Agnosticism & Model Switching

LLM integration is decoupled behind an `ILlmService` interface. Concrete implementations interface with OpenAI-compatible REST endpoints via the official .NET OpenAI SDK. Configuration parameters (`Provider`, `BaseUrl`, `ApiKey`, `ModelName`, `Temperature`) are bound from `appsettings.json` or environment variables. This enables zero-code switching between Iranian gateways such as GapGPT, native OpenAI endpoints, or local Ollama instances.

3.4 Deterministic Post-Processing (TextSanitizer)

Empirical testing revealed an LLM tendency to wrap Markdown and SQL deliverables in outer code fences (e.g., ```` ```markdown ... ``` ````), which breaks file-system viewers and collapses embedded Mermaid diagrams. The architecture introduces `TextSanitizer.StripOuterCodeFence()` as a deterministic pipeline interceptor. Using compiled regular expressions, it strips only the outermost code fence wrapper upon file generation while preserving nested ```` ```mermaid ```` and ```` ```sql ```` blocks.

4. Automated Live DBMS Validation (+15% Bonus Engine)

The validation engine connects directly to a live PostgreSQL 16 container hosted via Docker. Implemented in `PostgresValidator`, it executes `schema.sql` followed immediately by `seed.sql` using managed `Npgsql` connections.

> **Self-Healing Execution Loop:** When a database exception occurs (e.g., syntax error, table ordering deadlock, or foreign key violation), the engine intercepts `PostgresException`, extracting the exact SQL error code, line number, and constraint details. This feedback is passed into `stage6_sql_repair.scriban`, directing the model to repair both scripts simultaneously. The repaired scripts are written back to `output/` and re-tested up to a configurable maximum of 3 attempts. All diagnostic traces and repairs are logged in `logs/llm_calls.jsonl`.

5. Known Limitations & Future Work

- **Context Window Saturation on Massive Schemas:** While highly performant for architectures with 10–25 entities, enterprise schemas spanning 100+ tables could exceed prompt context limits during Stage 5 multi-file revision. Future versions could partition schemas into domain-driven sub-graphs.
- **Complex Circular Foreign Key Resolution:** If two tables possess mutually mandatory foreign key dependencies, standard single-pass `INSERT` statements in `seed.sql` will fail. The current generator mitigates this by ordering nullable references last, but a fully automated schema should inject deferred constraints (`DEFERRABLE INITIALLY DEFERRED`).
- **Multi-Node Connection Pooling:** The validation engine currently tests against a single isolated database node. Extending validation to read-replica topologies or multi-master replication clusters remains an architectural opportunity.

6. Conclusion

The developed DBA Agent demonstrates that combining structured C# state orchestration, externalized prompt engineering, and live database self-healing creates a dependable bridge between natural language problem statements and verifiable relational database schemas. The resulting deliverables satisfy all criteria set forth in Dr. Manavi's specification and provide an end-to-end framework for reliable database design automation.